Project Synopsis

FairView: An AI-Powered Bias Detector for Employee Performance Evaluation

AIML Mini Project | Department of Computer Engineering & Applications (AIML), GLA University
Project category: Software (web application with a machine learning module)

Cover

- **Project title:** FairView: An AI-Powered Bias Detector for Employee Performance Evaluation
- **Project type:** Software (MERN web application + Python ML service)
- **Team ID:** 123 (single-member team)
- **Team member:** Abhishek Goswami, Roll No. 12515500018
- **Institute / Course:** GLA University / B.Tech CSE (AIML)
- **Mentor:** Not assigned yet
- **Version:** v2.0
- **Date:** 22 Oct 2026
- **GitHub repository link:** https://github.com/abhishekgoswami0720/FairView-An-AI-Powered-Bias-detector

Revision history

Version	Date	Author	Change
v0.1	28 Sep 2026	Abhishek Goswami	Project idea proposal prepared
v1.0	02 Oct 2026	Abhishek Goswami	Synopsis prepared with market research and competitor analysis
v2.0	22 Oct 2026	Abhishek Goswami	Updated with current progress (backend completed) and the blind rating rule

1. Overview

1.1 Problem statement

In most organisations the reporting manager assigns work to an employee and also decides that employee's performance rating. Nobody independently checks whether the rating matches the actual quality of the work. A manager who favours some employees can rate them higher than others whose work is equally good, and the affected employee has no evidence to show this. Over time this affects appraisals, promotions and employee morale.

1.2 Problem definition

The underlying issue is that task assignment, evaluation of the work and the final rating are all controlled by one person. Current HR software such as Workday and Lattice moves the appraisal process online, and some of these tools compare a manager's ratings with those of other managers. All of these checks, however, use data from inside the company. None of them compares a manager's rating with an independent rating of the same piece of work.

1.3 Goal

Build a working web platform in which the client who receives the work also rates it, the manager cannot change that rating, and a machine learning module compares the two ratings to flag managers whose ratings differ consistently from the client's for particular employees.

1.4 Non-goals

- Taking any automatic action against a manager. The system only flags cases for HR to review.
- Detecting bias in the way tasks are distributed, for example giving easier work to a favoured employee.
- Storing protected attributes such as gender, caste, religion or age.
- Salary, promotion and payroll management.

1.5 Value proposition

The client sits outside the company and has no stake in internal office relationships, so the client's rating can act as an independent reference point. Comparing against it can also catch a manager who rates most of the team fairly but consistently under-rates one person. This pattern does not show up when managers are only compared with each other.

1.6 Current project status

Development started on 6 October 2026 and the project is currently in week 3 of 12. The backend is complete and tested. The frontend and the ML module are yet to be built.

Module	Status	Remarks
Requirements and database design	Done	Schema and API list prepared
Login and role-based access	Done	Four roles; only HR can create new accounts
Projects, tasks and work share	Done	Total work share in a project cannot exceed 100%
Locked ratings and audit log	Done	Edit and delete requests are refused and logged
Backend tests	Done	88 automated tests (Jest + Supertest), all passing
React frontend	Next	Weeks 4-5
ML module	Planned	Weeks 6-9

1.7 Expected outcome

At the end of the project the following will be delivered: a working web application with separate dashboards for clients, managers, employees and HR; an ML module that gives each manager a fairness score and flags unusual rating patterns; and a test report on synthetic data showing precision, recall and the false-flag rate for fair managers.

2. Scope and Control

2.1 In-scope

- Login with role-based access for Client, Manager, Employee and HR.
- Client: create projects and rate delivered projects.
- Manager: split projects into tasks, assign them with a work share, rate each task and deliver the project.
- Employee: update task status, submit work and view client ratings.
- HR: create user accounts and review flagged cases.
- ML module for rating gap analysis and fairness scoring.
- Audit log of all rating actions.

2.2 Out-of-scope

- Mobile application.

- Payroll, salary and promotion workflows.
- Integration with existing HRMS tools such as Workday.
- Multi-company (SaaS) support and single sign-on.

2.3 Assumptions

- Each project has an external client who receives the delivered work.
- Clients are willing to give a rating from 1 to 10 and a short review.
- Managers rate every task on the same 1 to 10 scale.
- At least 10 rated tasks are needed before a manager-employee pair is judged.

2.4 Constraints

- Single-member team working part-time along with regular coursework.
- Timeline of 12 weeks.
- No real company appraisal data is available, so the ML module will be developed on synthetic data and a small survey.
- Free-tier hosting with limited compute.

2.5 Dependencies

- MongoDB Atlas (free tier) for the database.
- Pre-trained sentiment model from Hugging Face.
- Docker, GitHub and GitHub Actions; Vercel and Render for hosting.

2.6 Acceptance criteria and sign-off

- GIVEN a client has rated a project WHEN any user tries to edit or delete the rating THEN the request is refused and the attempt is recorded in the audit log.
- GIVEN a manager has not rated every task WHEN the manager tries to deliver the project THEN delivery is blocked.
- GIVEN at least 10 rated tasks with a large repeated gap WHEN the fairness check runs THEN the case appears on the HR dashboard with its rating history.
- GIVEN fewer than 10 rated tasks WHEN the fairness check runs THEN the result is "not enough data" and no flag is raised.
- Sign-off: the mentor approves the final demonstration and the test report.

Sign-off table

Stakeholder	Role	Decision area	Signature / Approval	Date
Project mentor	Mentor	Scope, final acceptance	Not assigned	
Abhishek Goswami	Team Leader	Release readiness		
Department	Approving authority	Start of implementation		

3. Stakeholders and RACI

Activity	Responsible (R)	Accountable (A)	Consulted (C)	Informed (I)
Requirements	Abhishek	Abhishek	Mentor	Department
Database and API design	Abhishek	Abhishek	Mentor	-
Backend development	Abhishek	Abhishek	-	Mentor

Activity	Responsible (R)	Accountable (A)	Consulted (C)	Informed (I)
Frontend development	Abhishek	Abhishek	-	Mentor
ML module and evaluation	Abhishek	Abhishek	Mentor	-
Testing	Abhishek	Abhishek	Mentor	-
Documentation	Abhishek	Abhishek	Mentor	Department

Since this is a single-member project, all activities are carried out by the team leader. Once a mentor is assigned, progress will be reviewed with the mentor at the checkpoints in weeks 4, 7 and 10.

4. Team and Roles

Member	Role	Roll No.	Responsibilities
Abhishek Goswami	Team Leader (single member)	12515500018	Requirements, backend, frontend, ML module, testing and documentation

5. Week-wise Plan and Assignments

Plan for a 12-week build starting 6 Oct 2026. The core backend was finished in week 3, slightly ahead of plan, so frontend work moves forward by one week.

Week	Dates	Milestone	Main tasks	Deliverable	Status
1	6-12 Oct	Requirements and design	Feature list, database schema, API list	Design notes	Done
2	13-19 Oct	Authentication	JWT login, four roles, access control	Working login	Done
3	20-26 Oct	Core backend	Projects, tasks, work share, locked ratings, audit log, tests	Backend + 88 tests	Done
4-5	27 Oct-9 Nov	Frontend	React dashboards for all four roles	v0.5 (without ML)	Next
6	10-16 Nov	Dataset	Synthetic data generator, survey form	Dataset v1	Planned
7	17-23 Nov	Baseline model + checkpoint	Gap and z-score analysis, first evaluation	v0.7, evaluation note	Planned
8	24-30 Nov	ML service	FastAPI service connected to backend	Integrated ML service	Planned
9	1-7 Dec	Isolation Forest + HR view	Multi-feature model, flagged-case list	Ranked flag list	Planned
10	8-14 Dec	Sentiment check + checkpoint	Feedback tone analysis; final scope decision	Checkpoint note	Planned
11	15-21 Dec	Testing	End-to-end tests, user testing	Test report	Planned
12	22-28 Dec	Final demo	Fixes, report and presentation	v1.0, final report	Planned

6. Users and Use

6.1 Intended users

- **Client:** the customer who receives the delivered project. Needs a quick way to rate the work.
- **Manager:** assigns and rates tasks. Needs simple task management and can see their own fairness score.
- **Employee:** works on tasks. Needs a record of client ratings that the manager cannot change.
- **HR:** reviews flagged cases and needs the rating history behind each flag.

6.2 Main user journeys

- Client: log in → open delivered project → give rating (1-10) and review → submit (rating is locked).
- Manager: log in → split project into tasks → assign with work share → rate submitted tasks → deliver project.
- Employee: log in → view tasks → start task → submit work → view client rating.
- HR: log in → open flagged cases → check rating history → decide on follow-up.

6.3 Usability

- Rating a project should take the client less than one minute.
- Flags are shown with a text label and an icon, not colour alone.

7. Market Research and Existing Solutions

7.1 Market size

According to Grand View Research, the global employee performance management software market was valued at about USD 4.19 billion in 2025 and is expected to reach about USD 10.52 billion by 2033, a CAGR of about 12.3%. Estimates from other research firms vary widely (roughly USD 3.6 to 13.4 billion), so these figures are only an indication of size.

7.2 Target users and market segment

FairView needs a client to compare with, so it is aimed at organisations where work is delivered to an external client: IT services and consulting companies, design and marketing agencies, and BPO companies. Indian IT services firms, where most employees work on client projects, are the first target segment.

7.3 Existing solutions and competitor analysis

Product	Approach to bias	Compared against	Limitation
Workday	Bias reports by gender, age and tenure; calibration across levels	Other managers, group averages	No independent check on the quality of the work
Lattice	Flags rating differences between groups and between managers	Other managers, group averages	Only internal comparison
Culture Amp	Calibration sessions and outlier detection	Other managers	Only internal comparison
Confirm	Organisational network analysis of who works with whom	Colleague interaction data	Signal still comes from inside the company
Textio	Detects biased language while a review is written	Language patterns	Checks wording, not the score
PerformYard	Structured calibration process	Other managers	Process tool, not a detector

7.4 Positioning and differentiation

- **Unique angle:** the manager's rating is compared with an independent rating from the client, instead of only comparing managers with each other.
- **Practical value:** can detect a manager who is fair to most of the team but consistently unfair to one employee.
- **Limitation:** works only where an external client exists, so internal teams such as finance cannot use it.

7.5 Real-world applications

- IT services and consulting firms, for appraisals of employees working on client projects.
- Design and content agencies, where work goes directly to clients.
- Freelance platforms, where client ratings already exist.
- Colleges, to compare a project guide's marks with an external examiner's marks.

8. Objectives and Success Metrics

The figures below are initial targets. They will be reviewed after the first model results are available.

- **O1 Rating integrity:** 100% of edit or delete attempts on locked ratings are refused and logged. KPI: audit log check.
- **O2 Detection quality:** precision of at least 0.80 and recall of at least 0.75 on synthetic data with known bias. KPI: precision, recall.
- **O3 False flags:** no more than 5% of fair managers flagged. KPI: false-positive rate. This is the most important target, since a wrong flag can harm an honest manager.
- **O4 Explainability:** every flag shows the rating history behind it.
- **O5 Speed:** a client rating appears on the employee dashboard within 5 seconds.
- **O6 Usability:** at least 80% of test users can rate a project in under one minute.

9. Key Features

Feature	Description	Priority	Status
Role-based login	JWT login, four roles, account creation by HR only	Must	Built
Project and task flow	Client creates project; manager splits it into tasks with a work share	Must	Built
Locked client rating	Client rating cannot be edited or deleted by anyone	Must	Built
Blind manager rating	Manager must rate every task before delivery, before the client's rating exists	Must	Built
Audit log	Records every rating action and blocked attempt	Must	Built
Z-score gap analysis	Flags unusual gaps between manager and client ratings	Must	Planned
Fairness score	Score per manager with supporting rating history	Must	Planned
Isolation Forest	Multi-feature outlier detection	Should	Planned
HR review dashboard	List of flagged cases	Should	Planned
Sentiment check	Compares the tone of manager feedback with the rating	Could	Planned

The blind rating rule was added during development. If a manager could rate after seeing the client's score, they could copy it, and the comparison would then show nothing.

10. Architecture

10.1 High-level

- **Frontend:** React single-page application with a separate dashboard for each role.
- **Backend:** Node.js and Express REST API. Every request is checked for a valid JWT and the user's role.
- **Database:** MongoDB. The rating and audit log collections are write-once.
- **ML service:** Python FastAPI service that receives rating pairs from the backend and returns fairness scores. It is not exposed publicly.

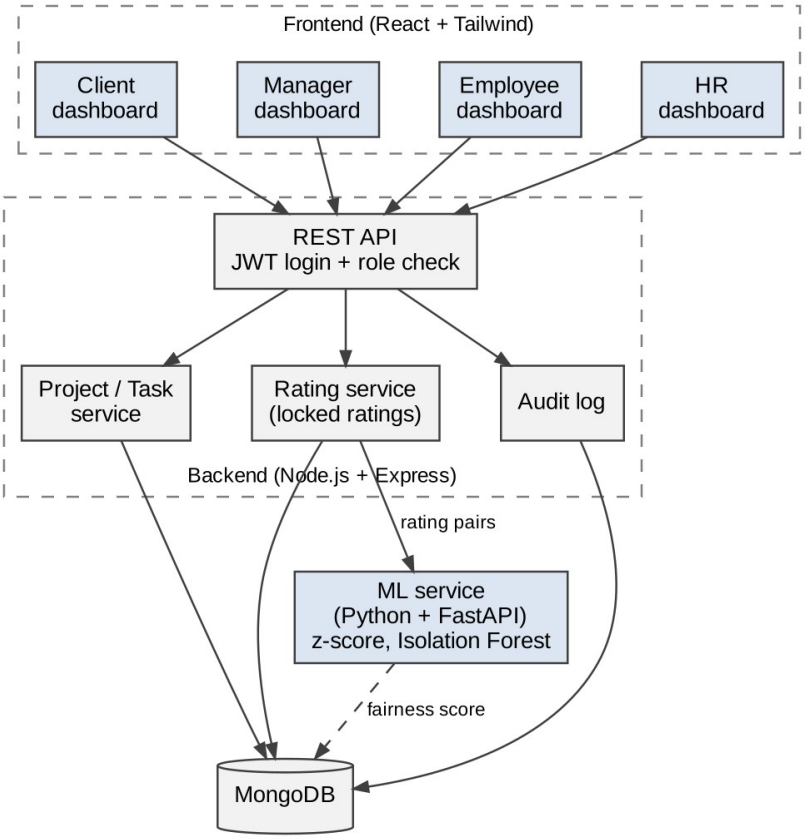


Figure 1: System architecture of FairView

10.2 Technology stack

Layer	Technology
Frontend	React.js, Tailwind CSS, Recharts
Backend / API	Node.js, Express.js, JWT, bcrypt
Database	MongoDB with Mongoose
ML service	Python, FastAPI
ML libraries	scikit-learn, pandas, NumPy, Hugging Face Transformers
Testing	Jest, Supertest, pytest, Playwright
Deployment	Docker, Vercel, Render, MongoDB Atlas
Version control	Git, GitHub, GitHub Actions

10.3 AI/ML models

Model	Use in FairView	Reason for choice
Z-score	Measures how unusual an employee's average rating gap is compared with the manager's normal gap. A value above 2 or below -2 is treated as unusual.	Simple, explainable, works with small data
Isolation Forest (scikit-learn)	Finds outlier manager-employee pairs using several features together	Unsupervised; no labelled bias data needed
DistilBERT sentiment model	Checks whether the tone of manager feedback matches the rating	Small enough for free hosting
TF-IDF + Logistic Regression	Fallback sentiment model	Lightweight if DistilBERT is too slow

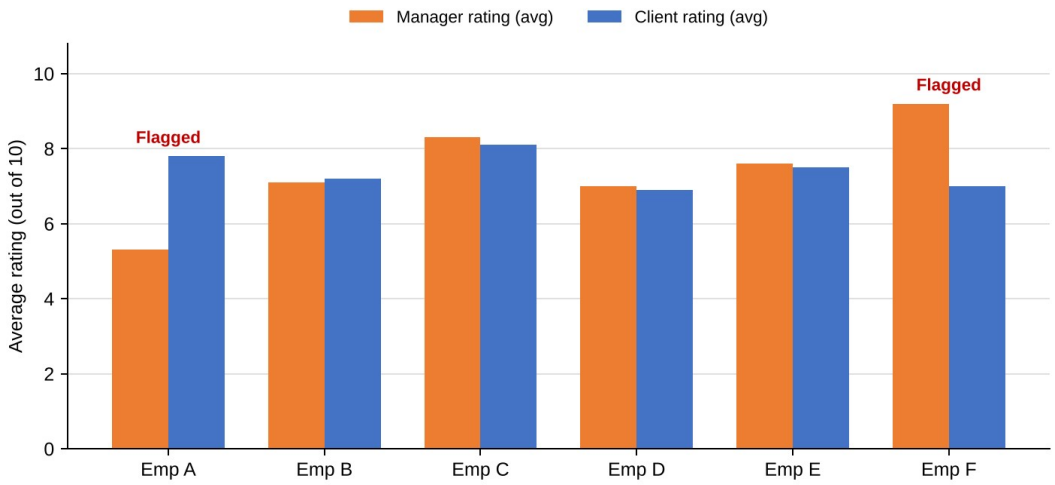


Figure 2: Illustrative output on synthetic data. Emp A is rated much lower and Emp F much higher by the manager than by the client, so both are flagged.

10.4 API snapshot

Endpoint	Method	Role	Purpose
/api/auth/login	POST	All	Log in and receive a JWT
/api/users	POST	HR	Create a user account
/api/projects	POST	Client	Create a project and choose its manager
/api/tasks	POST	Manager	Create a task with a work share
/api/tasks/:id/status	PATCH	Employee, Manager	Start, submit or send back for rework
/api/assessments	POST	Manager	Rate a submitted task (locked)
/api/projects/:id/deliver	POST	Manager	Deliver once all tasks are submitted and rated
/api/ratings	POST	Client	Rate a delivered project (locked)
/api/ratings/:id	PUT / PATCH / DELETE	None	Always refused (403) and logged

Endpoint	Method	Role	Purpose
/ml/analyse	POST	Backend only	Fairness analysis (planned)

10.5 Deployment strategy

- Frontend on Vercel; backend and ML service on Render; database on MongoDB Atlas.
- Each service is packaged with Docker so it runs the same locally and on the server.
- GitHub Actions runs the tests on every push before deployment. If a release breaks, the last working version is redeployed.
- Secrets such as the JWT secret and database URI are kept in environment variables, not in the code.

11. Data Design

11.1 Main entities

Entity	Key fields	Notes
User	name, email, passwordHash, role	Password stored only as a bcrypt hash
Project	client, manager, title, status	Status: open, delivered, rated
Task	project, employee, workShare, status	Work share cannot be changed after creation
ManagerAssessment	task, manager, score (1-10), feedback	Write-once
ClientRating	project, client, score (1-10), review	Write-once, one per project
AuditLog	actor, action, outcome, createdAt	Write-once
FairnessScore	manager, score, flags	Planned (week 7-9)

11.2 Linking a project rating to individual employees

The client rates the whole project, not individual employees. To compare ratings per employee, each task carries a work share that is fixed when the task is created. For example, if Employee A did 70% of a project and Employee B did 30%, and the client rated the project 8/10, then this rating is compared with both employees' manager ratings with weights of 0.7 and 0.3. Since work shares cannot be changed later, they cannot be adjusted to hide a gap. This is still an approximation and is listed under limitations.

11.3 Dataset

Dataset	Source	Content	Use	Notes
Synthetic ratings	Python generator script	Manager and client ratings; known bias added to some managers	Training and evaluation	Bias amount is controlled, so precision and recall can be measured
Survey responses	Google Form (target 50+)	Experience of unfair ratings from working professionals and interns	Checking realism of test cases	Anonymous and voluntary
Sample feedback text	Written for testing	Short manager comments	Sentiment module check	Used only if the sentiment module is built

11.4 Privacy, retention and backup

- Only name, work email and role are stored. No protected attributes are collected.
- Manager feedback is visible only to the manager who wrote it, the employee concerned and HR.
- Flags are visible only to HR and the flagged manager.
- A weekly export of the database is kept as a backup, and code is backed up on GitHub.

12. Technical Workflow Diagrams

12.1 Use case diagram

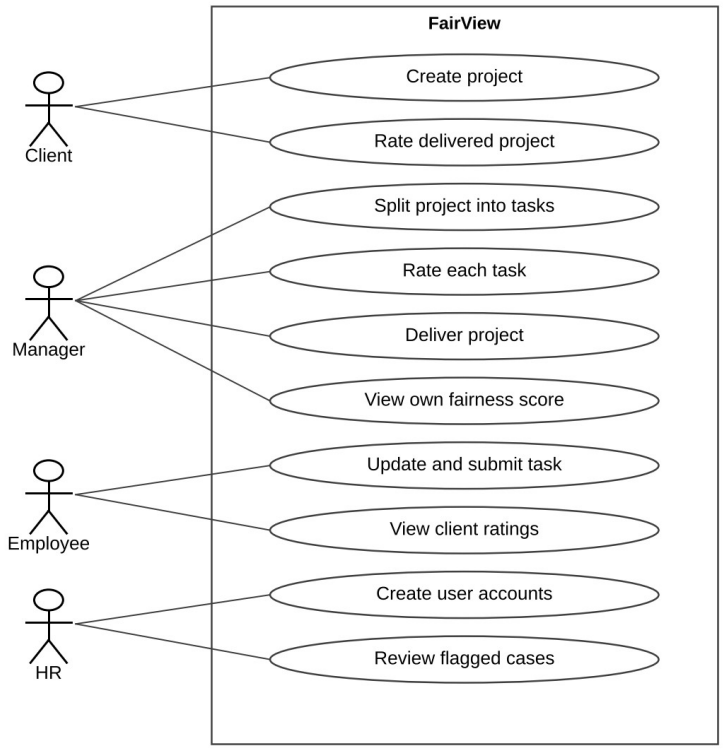


Figure 3: Use case diagram. No role has an action to edit or delete a rating.

12.2 State transition diagram

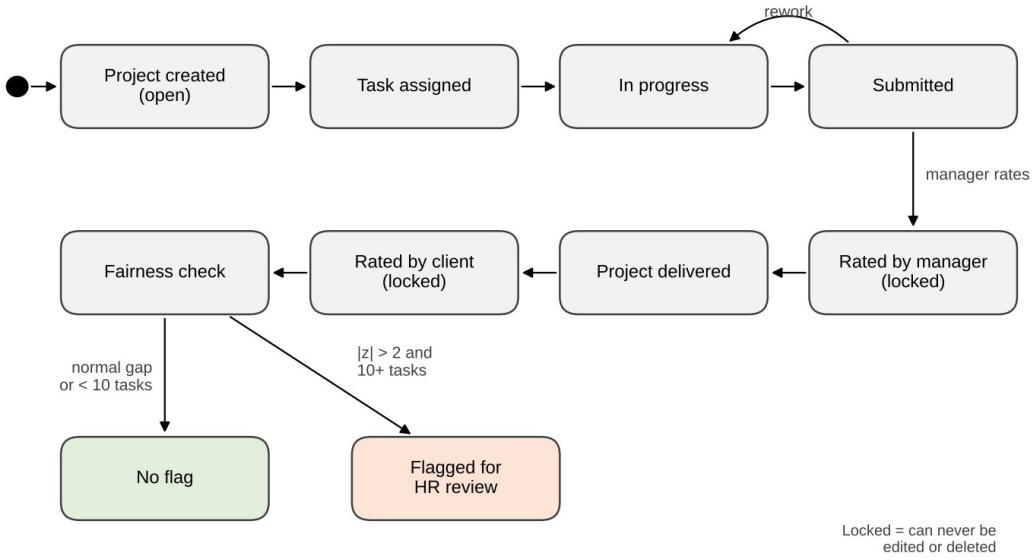


Figure 4: Life cycle of a project and its ratings

12.3 Sequence diagram

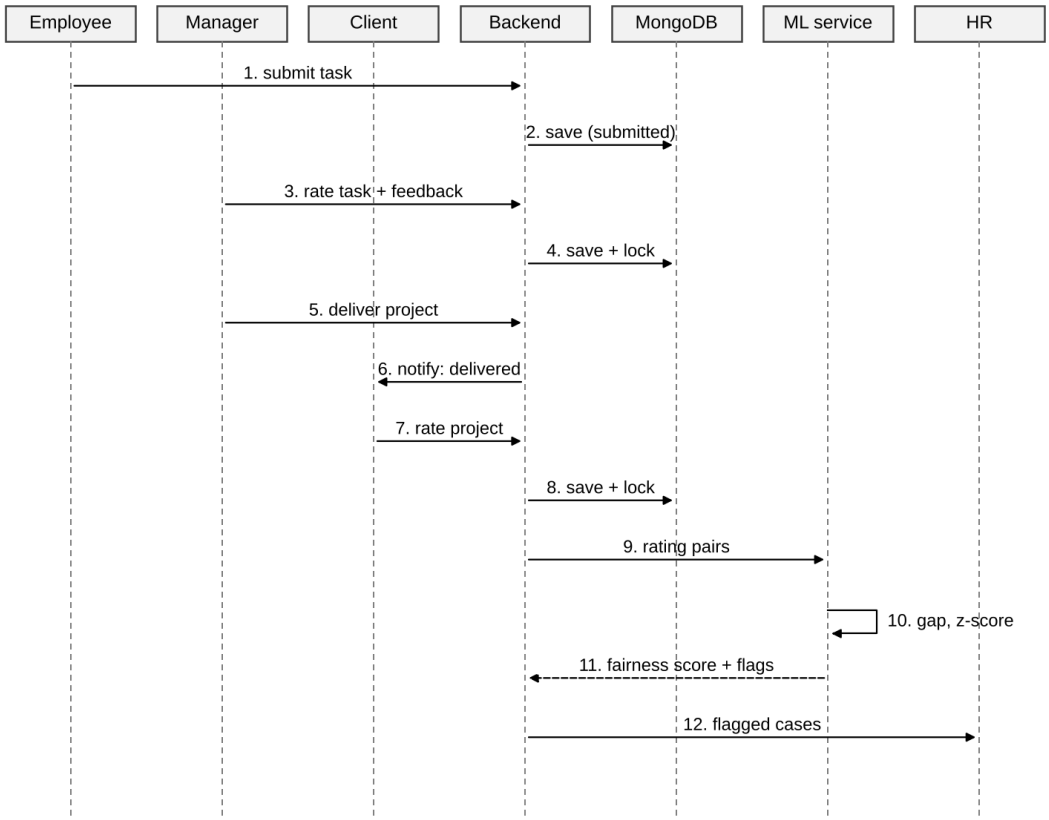


Figure 5: Order of messages from task submission to HR review

12.4 Data flow diagram (Level 1)

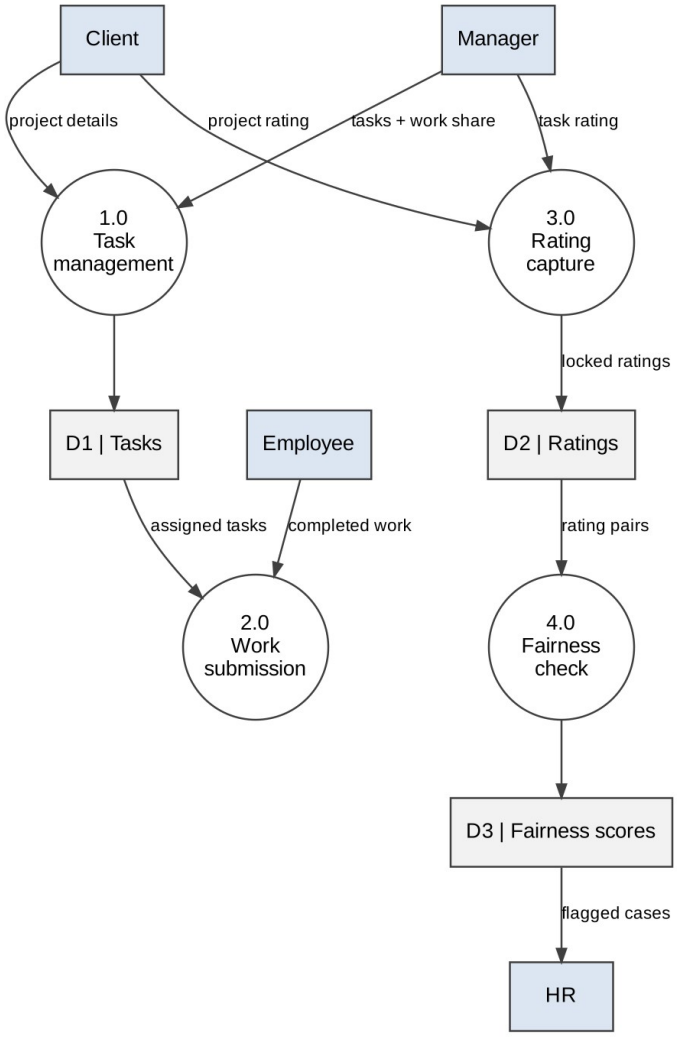


Figure 6: Level 1 data flow diagram

12.5 ER diagram

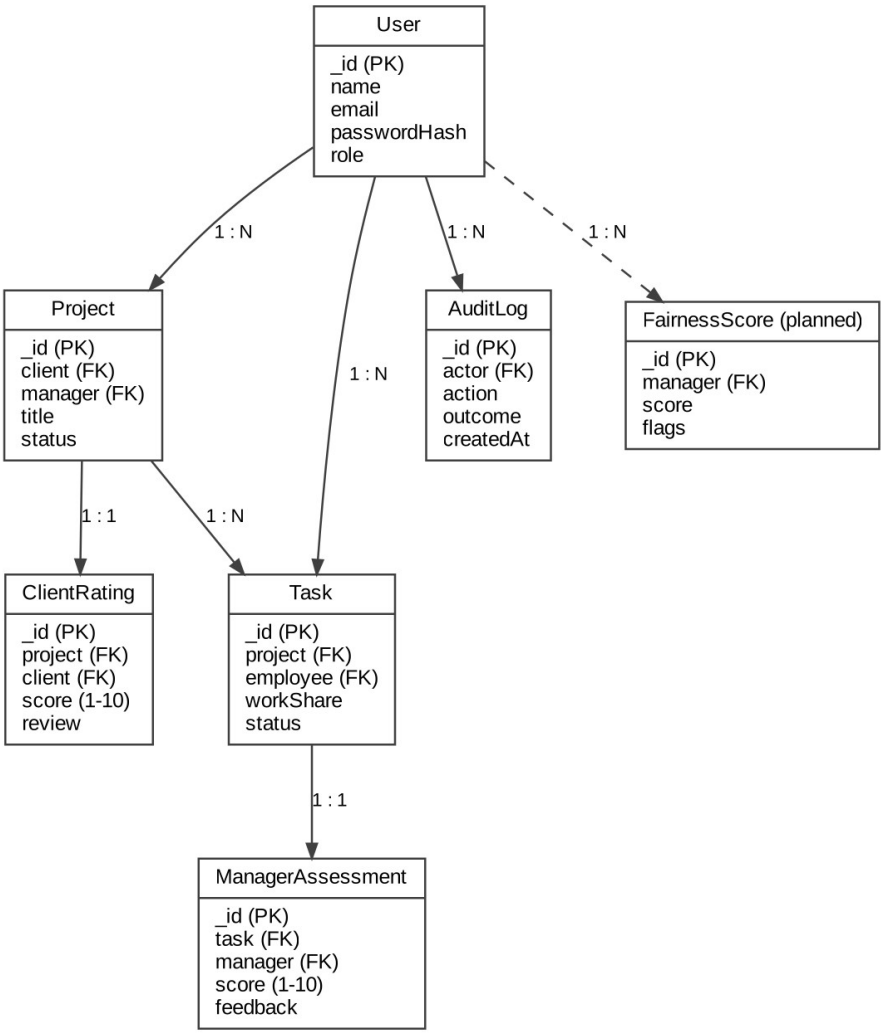


Figure 7: Entity relationship diagram (FairnessScore is planned)

12.6 ML workflow

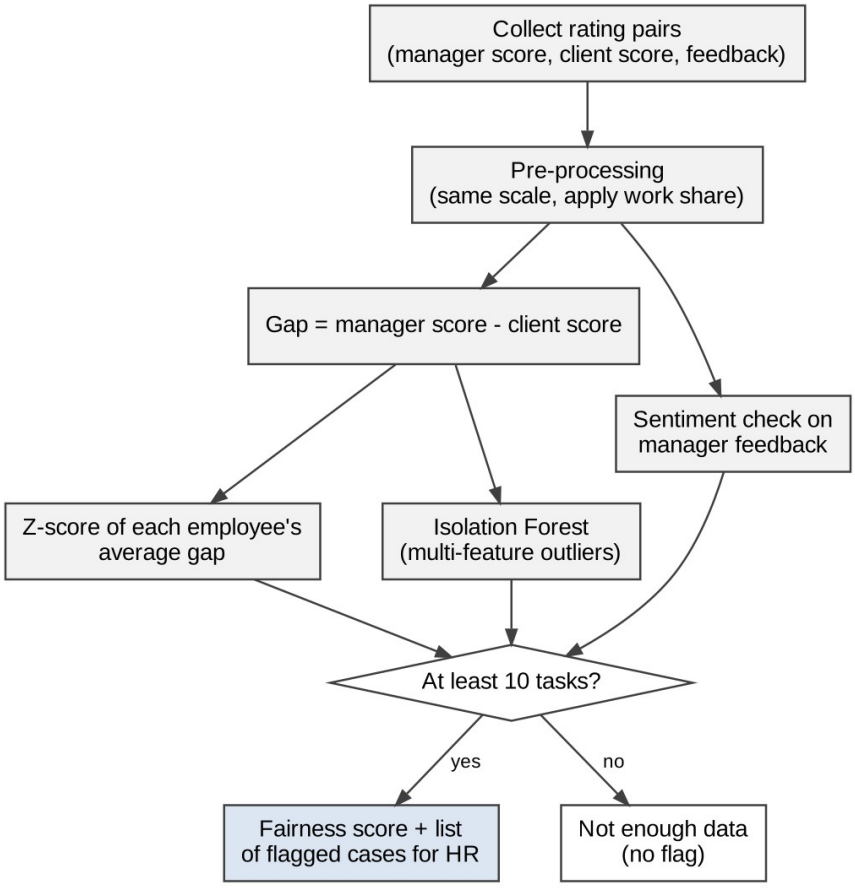


Figure 8: Steps of the fairness check in the ML service

13. Quality: NFRs and Testing

13.1 Non-functional requirements

Metric	Indicator	Target	Measurement
Availability	Uptime	99% during demo period	Uptime monitor
API latency	95th percentile response time	800 ms or less	Server logs
ML latency	Analysis time per manager	3 s or less	Service logs
Error rate	Failed requests	Below 1%	Server logs
Rating integrity	Successful edits of locked ratings	0	Audit log check

13.2 Test plan

Area	Type	Tools	Status
Locked ratings and roles	Unit and negative tests	Jest, Supertest	Passing
Work share and blind rating rules	Unit tests	Jest	Passing
ML module	Evaluation on synthetic data	pytest, scikit-learn	Week 7
Full user flow	End-to-end	Playwright	Week 11

Area	Type	Tools	Status
Usability	User testing, 3 students per role	Timed tasks, feedback form	Week 11

13.3 ML evaluation method

1. Generate synthetic data in which some managers are biased by a known amount, and check whether the system flags exactly those managers (precision and recall).
2. Reduce the amount of bias step by step to find the smallest gap the method can still detect.
3. Compare these gaps with the survey responses to check whether they are realistic.

Since the bias in step 1 is added by us, these results show that the method works. They do not show how it will perform on real company data.

13.4 Environments

Local machine (Docker Compose) → staging deployment on Vercel and Render → final demo deployment.

14. Security and Compliance

14.1 Threats and mitigations

Threat	Example	Mitigation
Rating tampering	Manager edits or deletes a client rating	Refused at API level and at database model level; attempt logged
Role misuse	User tries to act as HR	JWT with expiry, role check on every request, no public sign-up
Data leakage	Manager views another manager's score	Access limited by role and ownership
Misuse of flags	Flag treated as proof of bias	Flags marked "for review" and shown with full rating history
Direct database access	Admin edits data in MongoDB directly	Not prevented in this version; tamper-evident hashing is future work

14.2 Access control

Passwords are hashed with bcrypt. Login tokens (JWT) expire after 2 hours. Only HR can create new user accounts.

14.3 Logging

Every rating action and every blocked attempt is stored in a write-once audit log with the user, action, outcome and time.

14.4 Compliance

This is an academic project. Only synthetic data and anonymous, voluntary survey responses are used. No real employee data is collected or shared.

15. Delivery and Operations

15.1 Release plan

- v0.5 (week 5): web platform without the ML module.
- v0.7 (week 7): z-score analysis working. This is the minimum deliverable.
- v1.0 (week 12): final version with the features kept after the week 10 checkpoint.

15.2 Version control and rollback

Code is tracked in Git with tagged versions. If a change breaks the system, the last working tag is redeployed.

15.3 Monitoring

Metric	Threshold	Response
API latency	Above 1 second	Check slow database queries, revert last change
Error rate	Above 2%	Check logs, roll back
ML service	Not responding	Show last saved scores; rest of the platform keeps working
Audit log	Rating saved without a log entry	Stop rating writes and investigate

15.4 Communication plan

Weekly progress update to the mentor once assigned, and task tracking through GitHub issues.

16. Risks and Mitigations

16.1 Risk table

Risk	Probability	Impact	Mitigation	Status
Workload too high for one person	High	High	Fixed order of features to cut; checkpoints in weeks 4, 7 and 10	Open
Synthetic data does not reflect real bias	High	High	Survey to check realism; limitation stated in report	Open
Splitting a project rating per employee is approximate	High	Medium	Work share fixed at task creation	Open
Fair manager wrongly flagged	Medium	High	Minimum of 10 tasks; flags only for review	Open
Clients do not submit ratings	Medium	High	Rating takes under a minute	Open
Sentiment model too slow on free hosting	Medium	Medium	TF-IDF fallback model	Open

16.2 Fallback decision

Checkpoints are held at the end of weeks 4, 7 and 10. If the project is behind schedule, features are dropped in a fixed order: first the sentiment check, then Isolation Forest, and then the separate HR dashboard (flags would be shown on the manager dashboard instead). The web platform, locked ratings and z-score analysis will not be dropped, since together they form the minimum working project. Each decision and its reason will be recorded and shared with the mentor.

17. Research and Evaluation

- **Literature review:** the Behavioural Insights Team (2021) studied 360-degree feedback for 4,328 senior managers in a UK public sector organisation using text analysis. It found signs that women received less specific and more tentative feedback, and that asking reviewers to write better feedback did not remove the problem. This supports using an independent reference instead of relying on managers to correct themselves.
- **Evaluation plan:** track the metrics in Section 8 after each model version, and report results on synthetic data alongside the survey findings.
- **Limitations:** a rating gap is not proof of bias, since the manager may know about issues the client does not see; bias in task assignment is not detected; per-employee client scores are approximate; results on synthetic data may not carry over to real companies; the method needs an external client; and the rating lock works at application level, so direct database edits by an administrator are not prevented.
- **Future scope and scalability:** task difficulty scoring to detect unfair task assignment; peer reviews as a third rating source for teams without clients; tamper-evident hashing of ratings; integration with HRMS tools used in India such as Darwinbox and Keka; and use on freelance platforms. The ML service is stateless and can be scaled horizontally, and its workload grows with the number of managers rather than employees.

18. Appendices

18.1 Glossary

JWT (JSON Web Token), RBAC (role-based access control), CAGR (compound annual growth rate), z-score (distance from the average in standard deviations), Isolation Forest (unsupervised outlier detection algorithm), precision (share of flagged cases that are truly biased), recall (share of biased cases that are flagged), DFD (data flow diagram), ER (entity relationship).

18.2 References

1. Grand View Research, Employee Performance Management Software Market Report. https://www.grandviewresearch.com/industry-analysis/employee-performance-management-software-market-report
2. Behavioural Insights Team (2021), Gender bias and performance feedback: a randomised control trial. https://www.bi.team/publications/gender-bias-and-performance-feedback-a-randomised-control-trial/
3. Behavioural Insights Team (2021), full research report (PDF). https://www.bi.team/wp-content/uploads/2021/06/Performance-report.pdf
4. Lattice, Tips for Mitigating Bias in Performance Reviews. https://lattice.com/articles/performance-review-bias
5. Confirm, Enterprise Calibration Feature Comparison: Confirm vs Workday vs Lattice vs Culture Amp. https://www.confirm.com/compare/enterprise-calibration-feature-shootout
6. Confirm, Best Performance Calibration Software 2026. https://www.confirm.com/blog/best-performance-calibration-software-2026-top-8-tools
7. Windmill, AI Tools for Detecting Bias in Performance Reviews. https://gowindmill.com/resources/lists/ai-tools-detecting-bias-performance-reviews/
8. Betterworks, Top Performance Management Tools for Workday Users (2026). https://www.betterworks.com/magazine/8-top-performance-management-tools-for-workday-users
9. Unveiling Gender Bias in Large Language Models: Using Teacher's Evaluation in Higher Education as an Example (2024), arXiv:2409.09652. https://arxiv.org/pdf/2409.09652
10. scikit-learn, IsolationForest documentation. https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.IsolationForest.html
11. Hugging Face, Transformers documentation. https://huggingface.co/docs/transformers/index

12. FastAPI documentation. https://fastapi.tiangolo.com/
13. MongoDB, Data modelling documentation. https://www.mongodb.com/docs/manual/data-modeling/
14. React documentation. https://react.dev/
15. FairView project repository. https://github.com/abhishekgoswami0720/FairView-An-AI-Powered-Bias-detector

Note: references 5 and 6 are published by a competing vendor, so their claims about other products may be one-sided.